

Práctica N° 11: Transformaciones Avanzadas en el Espacio 3D Reflexión General y Distorsión (Shearing)

1. Objetivos

- Analizar, deducir e implementar la transformación geométrica de reflexión con respecto a planos principales y planos arbitrarios en el espacio tridimensional.
- Comprender el comportamiento de la transformación de distorsión o cizallamiento (*Shearing*) en 3D y su modelado matemático.
- Desarrollar efectos de deformación dinámica manipulando matrices e inyectando datos en tiempo real mediante Shaders en la GPU.

2. Conceptos Teóricos

2.1. Reflexión 3D sobre Planos Coordenados Principales

La reflexión genera una imagen de espejo de un objeto tridimensional con respecto a un plano de simetría. Cuando dicho plano coincide con los planos coordenados principales (XY , XZ o YZ), el signo de la coordenada ortogonal al plano se invierte. Las matrices homonimizadas de transformación son:

$$R_{xy} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}, \quad R_{xz} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & -1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}, \quad R_{yz} = \begin{bmatrix} -1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

2.2. Reflexión General sobre un Plano Arbitrario

Si la superficie de reflexión no se halla alineada con el origen ni con los planos principales, es mandatorio componer una secuencia de transformaciones para aplicar la operación de espejo de forma válida:

1. **Traslación:** Desplazar un punto perteneciente al plano arbitrario $P_0(x_0, y_0, z_0)$ hacia el origen de coordenadas usando el vector $t = [-x_0, -y_0, -z_0]^T$.
2. **Rotaciones de Alineación:** Girar el vector normal del plano hasta hacerlo coincidir con uno de los ejes principales (por ejemplo, el eje Z), logrando que el plano arbitrario coincida con el plano XY .
3. **Reflexión:** Aplicar la matriz de reflexión del plano correspondiente (ej. R_{xy}).
4. **Rotaciones Inversas:** Revertir los giros efectuados en el paso 2.
5. **Traslación Inversa:** Revertir el desplazamiento inicial llevando el plano a su sitio original.

La formulación matemática compacta se lee de la siguiente forma:

$$M_{ref_gen} = T(P_0) \cdot R^{-1} \cdot R_{reflexion} \cdot R \cdot T(-P_0)$$

2.3. Distorsión 3D (Shearing / Cizallamiento)

La distorsión modifica la geometría de un objeto desplazando sus puntos en una dirección paralela a un eje coordenado en una magnitud proporcional a sus otras coordenadas. A diferencia de las escalas, no preserva los ángulos rectos originales pero mantiene constante el volumen del objeto. Una matriz general de cizallamiento 3D se modela como:

$$Sh = \begin{bmatrix} 1 & sh_{xy} & sh_{xz} & 0 \\ sh_{yx} & 1 & sh_{yz} & 0 \\ sh_{zx} & sh_{zy} & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Donde, por ejemplo, sh_{xy} especifica el grado de deformación de la coordenada x con respecto a la componente y .

3. Desarrollo de la Práctica

Implementaremos una aplicación moderna en la cual construiremos matrices explícitas de reflexión y cizallamiento usando arreglos lineales, transmitiendo dichas propiedades al Vertex Shader.

3.1. Código del Vertex Shader (shaderadv.vert)

```
1 #version 333 core
2 layout (location = 0) in vec3 aPos;
3 layout (location = 1) in vec3 aColor;
4
5 out vec3 vertexColor;
6
7 uniform mat4 transformMatrix;
8 uniform mat4 view;
9 uniform mat4 projection;
10
11 void main() {
12     gl_Position = projection * view * transformMatrix * vec4(aPos, 1.0);
13     vertexColor = aColor;
14 }
```

Listing 1: Vertex Shader con Matrices Personalizadas

3.2. Código del Fragment Shader (shaderadv.frag)

```
1 #version 333 core
2 in vec3 vertexColor;
3 out vec4 FragColor;
4
5 void main() {
6     FragColor = vec4(vertexColor, 1.0);
7 }
```

Listing 2: Fragment Shader Avanzado

3.3. Código de la Aplicación en C++

Ejercicio 1: Tenemos una pirámide con los siguientes vértices $A1 = [40, 40, 40, 1]$, $B1 = [80, 30, 20, 1]$, $C1 = [40, 80, 40, 1]$, $D1 = [20, 20, 20, 1]$. Obtener la reflexión de la pirámide sobre el plano $Y = 10$;

```
1 #include <GL/glew.h>
2 #include <GLFW/glfw3.h>
3 #include <glm/glm.hpp>
4 #include <glm/gtc/matrix_transform.hpp>
5 #include <glm/gtc/type_ptr.hpp>
6 #include <iostream>
7
8 const GLuint WIDTH = 600, HEIGHT = 600;
9
10 // --- FUNCION MANUAL DE MULTIPLICACION DE MATRICES ---
11 void multiplicacionMM(float A[4][4], int FA, int CA, float B[4][4], int
    FB, int CB, float C[4][4]) {
12     if (CA == FB) { // para que se puedan multiplicar
13         for (int i = 0; i < FA; i++) {
14             for (int j = 0; j < CB; j++) {
15                 C[i][j] = 0; // inicializar
16                 for (int k = 0; k < CA; k++) {
17                     C[i][j] += A[i][k] * B[k][j];
18                 }
19             }
20         }
21     }
22 }
23
24 // --- SHADERS ---
25 const char* vertexShaderSource = "#version 330 core\n"
26 "layout (location = 0) in vec3 aPos;\n"
27 "uniform mat4 transform;\n"
28 "uniform mat4 view;\n"
29 "uniform mat4 projection;\n"
30 "void main() {\n"
31 "    gl_Position = projection * view * transform * vec4(aPos, 1.0);\n"
32 "}\n";
33
34 const char* fragmentShaderSource = "#version 330 core\n"
35 "out vec4 FragColor;\n"
36 "uniform vec3 objectColor;\n"
37 "void main() {\n"
38 "    FragColor = vec4(objectColor, 1.0);\n"
39 "}\n";
40
41 GLuint compileProgram() {
42     GLuint vs = glCreateShader(GL_VERTEX_SHADER);
43     glShaderSource(vs, 1, &vertexShaderSource, NULL);
44     glCompileShader(vs);
45
46     GLuint fs = glCreateShader(GL_FRAGMENT_SHADER);
47     glShaderSource(fs, 1, &fragmentShaderSource, NULL);
48     glCompileShader(fs);
49
50     GLuint prog = glCreateProgram();
51     glAttachShader(prog, vs);
52     glAttachShader(prog, fs);
```

```

53     glLinkProgram(prog);
54
55     glDeleteShader(vs);
56     glDeleteShader(fs);
57     return prog;
58 }
59
60 int main() {
61     if (!glfwInit()) return -1;
62     glfwWindowHint(GLFW_CONTEXT_VERSION_MAJOR, 3);
63     glfwWindowHint(GLFW_CONTEXT_VERSION_MINOR, 3);
64     glfwWindowHint(GLFW_OPENGL_PROFILE, GLFW_OPENGL_CORE_PROFILE);
65
66     GLFWwindow* window = glfwCreateWindow(WIDTH, HEIGHT, "Ejercicio 1:
Reflexion Y=10 Manual", NULL, NULL);
67     if (!window) { glfwTerminate(); return -1; }
68     glfwMakeContextCurrent(window);
69
70     glewExperimental = GL_TRUE;
71     if (glewInit() != GLEW_OK) return -1;
72
73     glEnable(GL_DEPTH_TEST);
74     GLuint program = compileProgram();
75
76     // --- DATOS DE VERTICES ---
77     float axesVertices[] = {
78         0.0f, 0.0f, 0.0f, 150.0f, 0.0f, 0.0f, // Eje X
79         0.0f, 0.0f, 0.0f, 0.0f, 150.0f, 0.0f, // Eje Y
80         0.0f, 0.0f, 0.0f, 0.0f, 0.0f, 150.0f // Eje Z
81     };
82
83     float pyramidVertices[] = {
84         40.0f, 40.0f, 40.0f, // A1
85         80.0f, 30.0f, 20.0f, // B1
86         40.0f, 80.0f, 40.0f, // C1
87         20.0f, 20.0f, 20.0f, // D1
88         40.0f, 40.0f, 40.0f // A1 (Cierre)
89     };
90
91     GLuint VAO_Axes, VBO_Axes, VAO_Pyramid, VBO_Pyramid;
92
93     glGenVertexArrays(1, &VAO_Axes);
94     glGenBuffers(1, &VBO_Axes);
95     glBindVertexArray(VAO_Axes);
96     glBindBuffer(GL_ARRAY_BUFFER, VBO_Axes);
97     glBufferData(GL_ARRAY_BUFFER, sizeof(axesVertices), axesVertices,
GL_STATIC_DRAW);
98     glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, 3 * sizeof(float), (
void*)0);
99     glEnableVertexAttribArray(0);
100
101     glGenVertexArrays(1, &VAO_Pyramid);
102     glGenBuffers(1, &VBO_Pyramid);
103     glBindVertexArray(VAO_Pyramid);
104     glBindBuffer(GL_ARRAY_BUFFER, VBO_Pyramid);
105     glBufferData(GL_ARRAY_BUFFER, sizeof(pyramidVertices),
pyramidVertices, GL_STATIC_DRAW);
106     glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, 3 * sizeof(float), (

```

```

void*)0);
107     glEnableVertexAttribArray(0);
108
109     GLint transformLoc = glGetUniformLocation(program, "transform");
110     GLint viewLoc = glGetUniformLocation(program, "view");
111     GLint projLoc = glGetUniformLocation(program, "projection");
112     GLint colorLoc = glGetUniformLocation(program, "objectColor");
113
114     glm::mat4 projection = glm::ortho(-150.0f, 150.0f, -150.0f, 150.0f,
-150.0f, 150.0f);
115
116     while (!glfwWindowShouldClose(window)) {
117         glClearColor(0.0f, 0.0f, 0.0f, 1.0f);
118         glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);
119
120         glUseProgram(program);
121
122         glm::mat4 view = glm::lookAt(glm::vec3(50.0f, 50.0f, 150.0f),
glm::vec3(0.0f, 0.0f, 0.0f), glm::vec3(0.0f, 1.0f, 0.0f));
123
124         glUniformMatrix4fv(projLoc, 1, GL_FALSE, glm::value_ptr(
projection));
125         glUniformMatrix4fv(viewLoc, 1, GL_FALSE, glm::value_ptr(view));
126
127         // --- MATRIZ IDENTIDAD (Para Ejes y Piramide Original) ---
128         float identity[4][4] = {
129             {1, 0, 0, 0},
130             {0, 1, 0, 0},
131             {0, 0, 1, 0},
132             {0, 0, 0, 1}
133         };
134
135         // Enviar matriz identidad usando GL_TRUE para transponerla
correctamente
136         glUniformMatrix4fv(transformLoc, 1, GL_TRUE, &identity[0][0]);
137
138         // Dibujar Ejes
139         glUniform3f(colorLoc, 1.0f, 1.0f, 0.0f); // Amarillo
140         glBindVertexArray(VAO_Axes);
141         glDrawArrays(GL_LINES, 0, 6);
142
143         // Dibujar Piramide Original
144         glPolygonMode(GL_FRONT_AND_BACK, GL_LINE);
145         glUniform3f(colorLoc, 1.0f, 0.0f, 0.0f); // Rojo
146         glBindVertexArray(VAO_Pyramid);
147         glDrawArrays(GL_TRIANGLE_STRIP, 0, 5);
148
149         // --- CALCULAR MATRIZ REFLEJADA MANUAMENTE ---
150         // PASO 1. Matriz de traslacion delta = -10 (Llevar plano Y=10
al origen)
151         float M1[4][4] = {
152             {1, 0, 0, 0},
153             {0, 1, 0, -10},
154             {0, 0, 1, 0},
155             {0, 0, 0, 1}
156         };
157
158         // PASO 2. Matriz de reflexion sobre el plano XZ (Y = 0)

```

```

159     float M2[4][4] = {
160         {1, 0, 0, 0},
161         {0, -1, 0, 0},
162         {0, 0, 1, 0},
163         {0, 0, 0, 1}
164     };
165
166     // PASO 3. Matriz de traslacion delta = +10 (Regresar el plano a
167     Y=10)
168     float M3[4][4] = {
169         {1, 0, 0, 0},
170         {0, 1, 0, 10},
171         {0, 0, 1, 0},
172         {0, 0, 0, 1}
173     };
174
175     float MT[4][4];
176     float M_Final[4][4];
177
178     // Multiplicar: MT = M2 * M1
179     multiplicacionMM(M2, 4, 4, M1, 4, 4, MT);
180     // Multiplicar: M_Final = M3 * MT
181     multiplicacionMM(M3, 4, 4, MT, 4, 4, M_Final);
182
183     // Dibujar Piramide Reflejada pasando la matriz calculada
184     manualmente
185     glUniformMatrix4fv(transformLoc, 1, GL_TRUE, &M_Final[0][0]);
186     glUniform3f(colorLoc, 0.0f, 1.0f, 0.0f); // Verde
187
188     glDrawArrays(GL_TRIANGLE_STRIP, 0, 5);
189
190     glPolygonMode(GL_FRONT_AND_BACK, GL_FILL);
191
192     glfwSwapBuffers(window);
193     glfwPollEvents();
194 }
195
196 glDeleteVertexArrays(1, &VAO_Axes);
197 glDeleteBuffers(1, &VBO_Axes);
198 glDeleteVertexArrays(1, &VAO_Pyramid);
199 glDeleteBuffers(1, &VBO_Pyramid);
200 glDeleteProgram(program);
201
202 glfwTerminate();
203 return 0;
204 }

```

Listing 3: Aplicación C++ con Matrices de Reflexión y Shearing

Ejercicio 2: Se tienen una pirámide con los siguientes vértices en coordenadas homogéneas: $A1 = [40, 20, 40, 1]$, $B1 = [60, 30, 20, 1]$, $C1 = [40, 50, 30, 1]$ y $D1 = [20, 30, 20, 1]$. Obtener el reflejo de la pirámide con respecto al plano $z = \sqrt{3}y$

```

1 #include <GL/glew.h>
2 #include <GLFW/glfw3.h>
3 #include <glm/glm.hpp>
4 #include <glm/gtc/matrix_transform.hpp>
5 #include <glm/gtc/type_ptr.hpp>
6 #include <iostream>

```

```

7
8 const GLuint WIDTH = 600, HEIGHT = 600;
9
10 // --- FUNCION MANUAL DE MULTIPLICACION DE MATRICES ---
11 void multiplicacionMM(float A[4][4], int FA, int CA, float B[4][4], int
    FB, int CB, float C[4][4]) {
12     if (CA == FB) {
13         for (int i = 0; i < FA; i++) {
14             for (int j = 0; j < CB; j++) {
15                 C[i][j] = 0;
16                 for (int k = 0; k < CA; k++) {
17                     C[i][j] += A[i][k] * B[k][j];
18                 }
19             }
20         }
21     }
22 }
23
24 // --- SHADERS ---
25 const char* vertexShaderSource = "#version 330 core\n"
26 "layout (location = 0) in vec3 aPos;\n"
27 "uniform mat4 transform;\n"
28 "uniform mat4 view;\n"
29 "uniform mat4 projection;\n"
30 "void main() {\n"
31 "    gl_Position = projection * view * transform * vec4(aPos, 1.0);\n"
32 "}\n";
33
34 const char* fragmentShaderSource = "#version 330 core\n"
35 "out vec4 FragColor;\n"
36 "uniform vec3 objectColor;\n"
37 "void main() {\n"
38 "    FragColor = vec4(objectColor, 1.0);\n"
39 "}\n";
40
41 GLuint compileProgram() {
42     GLuint vs = glCreateShader(GL_VERTEX_SHADER);
43     glShaderSource(vs, 1, &vertexShaderSource, NULL);
44     glCompileShader(vs);
45
46     GLuint fs = glCreateShader(GL_FRAGMENT_SHADER);
47     glShaderSource(fs, 1, &fragmentShaderSource, NULL);
48     glCompileShader(fs);
49
50     GLuint prog = glCreateProgram();
51     glAttachShader(prog, vs);
52     glAttachShader(prog, fs);
53     glLinkProgram(prog);
54
55     glDeleteShader(vs);
56     glDeleteShader(fs);
57     return prog;
58 }
59
60 int main() {
61     if (!glfwInit()) return -1;
62     glfwWindowHint(GLFW_CONTEXT_VERSION_MAJOR, 3);
63     glfwWindowHint(GLFW_CONTEXT_VERSION_MINOR, 3);

```

```

64     glfwWindowHint(GLFW_OPENGL_PROFILE, GLFW_OPENGL_CORE_PROFILE);
65
66     GLFWwindow* window = glfwCreateWindow(WIDTH, HEIGHT, "Reflexion
Plano Z = sqrt(3)Y", NULL, NULL);
67     if (!window) { glfwTerminate(); return -1; }
68     glfwMakeContextCurrent(window);
69
70     glewExperimental = GL_TRUE;
71     if (glewInit() != GLEW_OK) return -1;
72
73     glEnable(GL_DEPTH_TEST);
74     GLuint program = compileProgram();
75
76     // --- DATOS DE VERTICES ---
77     float axesVertices[] = {
78         0.0f, 0.0f, 0.0f, 150.0f, 0.0f, 0.0f, // Eje X
79         0.0f, 0.0f, 0.0f, 0.0f, 150.0f, 0.0f, // Eje Y
80         0.0f, 0.0f, 0.0f, 0.0f, 0.0f, 150.0f // Eje Z
81     };
82
83     // Piramide Ejercicio 3
84     float pyramidVertices[] = {
85         40.0f, 20.0f, 40.0f, // A1
86         60.0f, 30.0f, 20.0f, // B1
87         40.0f, 50.0f, 30.0f, // C1
88         20.0f, 30.0f, 20.0f, // D1
89         40.0f, 20.0f, 40.0f // A1 (Cierre)
90     };
91
92     GLuint VAO_Axes, VBO_Axes, VAO_Pyramid, VBO_Pyramid;
93
94     glGenVertexArrays(1, &VAO_Axes);
95     glGenBuffers(1, &VBO_Axes);
96     glBindVertexArray(VAO_Axes);
97     glBindBuffer(GL_ARRAY_BUFFER, VBO_Axes);
98     glBufferData(GL_ARRAY_BUFFER, sizeof(axesVertices), axesVertices,
GL_STATIC_DRAW);
99     glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, 3 * sizeof(float), (
void*)0);
100     glEnableVertexAttribArray(0);
101
102     glGenVertexArrays(1, &VAO_Pyramid);
103     glGenBuffers(1, &VBO_Pyramid);
104     glBindVertexArray(VAO_Pyramid);
105     glBindBuffer(GL_ARRAY_BUFFER, VBO_Pyramid);
106     glBufferData(GL_ARRAY_BUFFER, sizeof(pyramidVertices),
pyramidVertices, GL_STATIC_DRAW);
107     glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, 3 * sizeof(float), (
void*)0);
108     glEnableVertexAttribArray(0);
109
110     GLint transformLoc = glGetUniformLocation(program, "transform");
111     GLint viewLoc = glGetUniformLocation(program, "view");
112     GLint projLoc = glGetUniformLocation(program, "projection");
113     GLint colorLoc = glGetUniformLocation(program, "objectColor");
114
115     glm::mat4 projection = glm::ortho(-150.0f, 150.0f, -150.0f, 150.0f,
-150.0f, 150.0f);

```



```

116     glm::mat4 view = glm::rotate(glm::mat4(1.0f), glm::radians(30.0f),
117     glm::normalize(glm::vec3(3.0f, 3.0f, 3.0f)));
118
119     while (!glfwWindowShouldClose(window)) {
120         glClearColor(0.0f, 0.0f, 0.0f, 1.0f);
121         glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);
122
123         glUseProgram(program);
124
125         glUniformMatrix4fv(projLoc, 1, GL_FALSE, glm::value_ptr(
126         projection));
127         glUniformMatrix4fv(viewLoc, 1, GL_FALSE, glm::value_ptr(view));
128
129         // --- MATRIZ IDENTIDAD ---
130         float identity[4][4] = {
131             {1, 0, 0, 0},
132             {0, 1, 0, 0},
133             {0, 0, 1, 0},
134             {0, 0, 0, 1}
135         };
136
137         // Dibujar Ejes
138         glUniformMatrix4fv(transformLoc, 1, GL_TRUE, &identity[0][0]);
139         glUniform3f(colorLoc, 1.0f, 1.0f, 0.0f); // Amarillo
140         glBindVertexArray(VAO_Axes);
141         glDrawArrays(GL_LINES, 0, 6);
142
143         // Dibujar Piramide Original
144         glPolygonMode(GL_FRONT_AND_BACK, GL_LINE);
145         glUniform3f(colorLoc, 1.0f, 0.0f, 0.0f); // Rojo
146         glBindVertexArray(VAO_Pyramid);
147         glDrawArrays(GL_TRIANGLE_STRIP, 0, 5);
148
149         // --- CALCULO MANUAL: REFLEXION SOBRE PLANO Z = sqrt(3)Y ---
150
151         // PASO 1: Rotacion de +30 sobre X (Alinea el plano con el plano
152         XZ, donde Y=0)
153         // cos(30) = 0.866025, sin(30) = 0.5
154         float M1[4][4] = {
155             {1.0f, 0.0f, 0.0f, 0.0f},
156             {0.0f, 0.866025f, -0.5f, 0.0f},
157             {0.0f, 0.5f, 0.866025f, 0.0f},
158             {0.0f, 0.0f, 0.0f, 1.0f}
159         };
160
161         // PASO 2: Reflexion sobre el plano XZ (Invierte el eje Y)
162         float M2[4][4] = {
163             {1.0f, 0.0f, 0.0f, 0.0f},
164             {0.0f, -1.0f, 0.0f, 0.0f},
165             {0.0f, 0.0f, 1.0f, 0.0f},
166             {0.0f, 0.0f, 0.0f, 1.0f}
167         };
168
169         // PASO 3: Rotacion de -30 sobre X (Devuelve el plano a su
170         inclinacion original)
171         // cos(-30) = 0.866025, sin(-30) = -0.5
172         float M3[4][4] = {
173             {1.0f, 0.0f, 0.0f, 0.0f},

```

```

170         {0.0f, 0.866025f, 0.5f, 0.0f},
171         {0.0f, -0.5f, 0.866025f, 0.0f},
172         {0.0f, 0.0f, 0.0f, 1.0f}
173     };
174
175     float MT[4][4];
176     float M_Final[4][4];
177
178     // Multiplicar: MT = M2 * M1
179     multiplicacionMM(M2, 4, 4, M1, 4, 4, MT);
180     // Multiplicar: M_Final = M3 * MT
181     multiplicacionMM(M3, 4, 4, MT, 4, 4, M_Final);
182
183     // Dibujar Piramide Reflejada
184     glUniformMatrix4fv(transformLoc, 1, GL_TRUE, &M_Final[0][0]); //
GL_TRUE transpone la matriz
185     glUniform3f(colorLoc, 0.0f, 1.0f, 0.0f); // Verde
186
187     glDrawArrays(GL_TRIANGLE_STRIP, 0, 5);
188
189     glPolygonMode(GL_FRONT_AND_BACK, GL_FILL);
190
191     glfwSwapBuffers(window);
192     glfwPollEvents();
193 }
194
195 glDeleteVertexArrays(1, &VAO_Axes);
196 glDeleteBuffers(1, &VBO_Axes);
197 glDeleteVertexArrays(1, &VAO_Pyramid);
198 glDeleteBuffers(1, &VBO_Pyramid);
199 glDeleteProgram(program);
200
201 glfwTerminate();
202 return 0;
203 }

```

Listing 4: Aplicación C++ con Matrices de Reflexión y Shearing

4. Actividades / Ejercicio Propuesto

A partir de la plantilla modular de código provista, desarrolle la siguiente actividad de laboratorio obligatoria:

- A) Realice la transformación de reflexión de una pirámide sobre el plano $3x + 5y + 6z = 0$.
- B) Implemente la reflexión geométrica estricta de la pirámide sobre el plano infinito $Z = 5,0$. Recuerde deducir y acoplar el término de traslación correspondiente dentro del cálculo de la matriz.
- C) **Distorsión Dinámica Cruzada:** Aplique sobre la pirámide reflejada una distorsión simultánea bidireccional donde la componente Y sea deformada por un factor proporcional a Z ($sh_{yz} = \cos(\text{tiempo})$) y la componente X sea deformada proporcionalmente a Y ($sh_{xy} = \sin(\text{tiempo})$).